ferryGL: Token-Free Automation for Syncing Cross-Organization GitHub Work into Linear

One-way, least-privilege sync that runs without a language model, for people and AI agents alike

Oussama Batouche

oussama.batouche@qutwo.com

White paper, version 1.0 • October 2026 • describes ferryGL 0.2.1

Abstract

AI agents can now do much of the routine work around software projects, but every run of an agent costs model tokens, and routine work recurs. This paper argues for *token-free automation*: deterministic programs that do a routine, rule-based task without a language model, and that people and agents call instead of doing the task themselves. It presents ferryGL, an open-source tool for one such task. Developers who work in another organization's repositories, such as a client's, often plan that work in their own team's Linear workspace, which Linear's GitHub integration cannot reach without the other organization's administrators. ferryGL copies the GitHub issues and pull requests a person is part of, one way, into that workspace, using only the person's own credentials. It keeps no state of its own: the links it attaches to Linear issues record the GitHub state it last acted on, so a status moves only when that state changes, and edits made by hand stay. A pure planner separates every read from every write, so every run can be previewed. In a randomized simulation of 2,000 histories (74,139 runs, with 5 % of writes failing), every safety property held in all 413,071 of its checks, and a metamorphic check exposed one order-dependence defect. Measured workloads, priced at published rates under assumptions that bracket how an agent behaves, show that an agent doing the job itself would process 91,620 to 217,857 tokens on a run of a 100-item route that finds a few changes, and that delegating the job to ferryGL cuts its cost 7 to 21 times across 24 models from 10 providers. Under the middle-estimate assumptions, a team of 100 syncing twice a workday would spend $596 to $59,576 a year with agents doing the job, $59 to $6,249 with agents delegating it, and nothing on inference with ferryGL alone.

Keywords: token-free automation, AI agents, inference cost, deterministic automation, one-way synchronization, idempotence, edge-triggered reconciliation, least privilege, GitHub, Linear

Cite as: Oussama Batouche. *ferryGL: Token-Free Automation for Syncing Cross-Organization GitHub Work into Linear.* White paper, version 1.0, October 2026. doi:10.5281/zenodo.23475998

1 Introduction

Many developers do their work in repositories that their own organization does not own. A consultancy's engineers work in their clients' repositories, a platform team contributes fixes upstream to the open-source projects it depends on, and partners send pull requests to each other [30, 31]. Their own team still plans in its own issue tracker, where it records what needs doing, who is doing it and how far it has come [9]. The work happens in one organization's repositories and is planned in another's tracker. This paper is about one instance of that gap: work on GitHub, tracked in Linear. Linear's GitHub integration links pull requests and issues to Linear issues, but only for repositories whose GitHub organization has connected it. An organization owner must make that connection (or, for single repositories, a repository administrator), and "it is not possible to connect multiple Linear workspaces to a single GitHub organization" [41]. A client's organization is often already connected to the client's own Linear workspace, if it uses Linear at all (Figure 1).

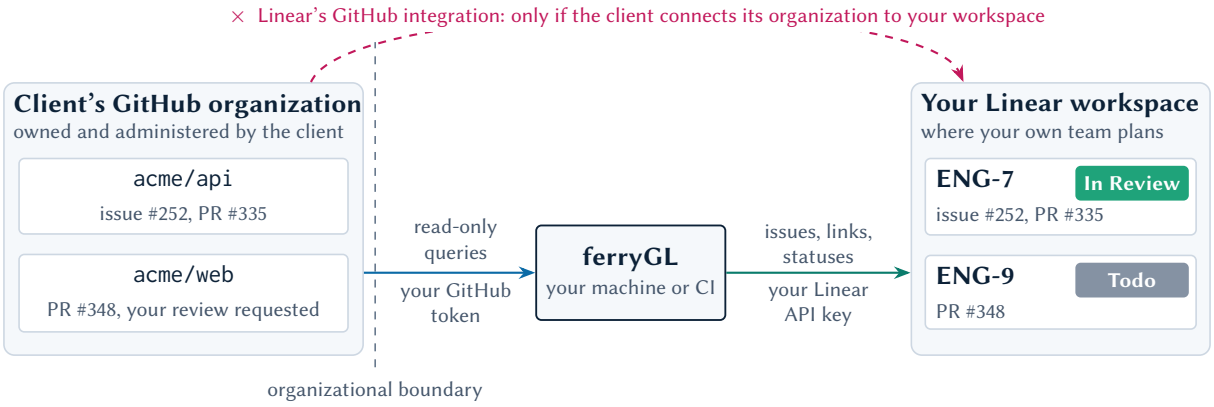


Figure 1. The gap ferryGL fills. Linear's GitHub integration needs the client's organization to connect it, and a GitHub organization can be connected to only one Linear workspace. ferryGL instead reads GitHub with the user's own token, read-only, and writes to the user's own Linear workspace with their own API key. Repository names and issue numbers here and throughout the paper are synthetic.

People bridge the gap in three ways today. They copy issues and pull requests by hand, and move each status again whenever something changes on GitHub: toil, in the vocabulary of site reliability engineering [54], which adds to the switching between tools that fragments developers' days [44]. They ask the client to connect its organization, which is often not theirs to ask. Or they ask a general-purpose AI agent, since agents that reason and call tools can drive both APIs [66, 67]. But the work recurs: an agent asked to keep two systems in step pays to read both again on every run, whether or not anything changed, and its behavior is not guaranteed to be the same twice [6, 51].

This paper argues that such work belongs in *token-free automation*: deterministic programs that do a routine, rule-based task without running a language model, and that people and agents call instead of doing the task themselves. The agent keeps the judgment, such as when to run the task and what to do about an exception, and the program does the repetition, at the cost of API requests rather than inference. ("Token-free" refers to model tokens throughout; such a program still authenticates with access tokens.)

ferryGL is such a program: a small command-line tool that a person, or their agent, runs by hand or on a schedule. Using only that person's GitHub token and Linear API key, it gives the issues and pull requests they are part of Linear issues assigned to them and linked to GitHub, and keeps their statuses in step. It never writes to GitHub, never touches a teammate's Linear issue, never deletes anything, and shows every change before making it. Its design rests on one decision: *the destination is the database.* The link on each Linear issue that ferryGL manages records the GitHub state ferryGL last acted on. Linear can find links by URL, so ferryGL needs no state of its own; and because it compares GitHub with the recorded state rather than with the Linear status, it moves a status only when something happens on GitHub, and leaves the user's own edits alone in between.

Contributions. This paper makes four contributions:

1. A design for personal, cross-organization, one-way issue synchronization under least privilege. It keeps its state in the destination's link metadata and acts on edges in the source's state, which gives idempotent reruns without a database and preserves manual edits (Sections 2 and 3).
2. An implementation whose safety rests on eight explicit invariants, a pure planner, and a client interface that cannot express the forbidden writes (Section 4).
3. An evaluation of the tool: a randomized, model-based simulation with injected write failures, which confirms its safety properties and exposes one defect; a request-cost model, validated against a mock of GitHub's API; and a planner benchmark (Sections 6.1 to 6.3).

4. A method to compare an agent that does a routine task itself, an agent that delegates it to a token-free tool, and the tool alone: measure the task's reads and writes, then price them under assumptions that bracket how an agent behaves. On ferryGL's task, across 24 models from 10 providers, delegating cuts an agent's cost 7 to 21 times on a route of 100 items, and more on larger routes (Sections 5, 6.4 and 6.5).

The evaluation's scripts and data are released with the paper (Appendix B).

2 Background and problem statement

2.1 GitHub and Linear

On GitHub, an *issue* is open or closed; a closed issue has a reason: completed, not planned or duplicate. A *pull request* is open, closed or merged, and an open pull request may be a draft. A person can be part of an item as its author, as an assignee, as a reviewer who has submitted a review, as a requested reviewer, or as the author of one of its commits. A pull request can declare the issues it closes with keywords such as "Fixes #10" in its description [25], and search finds issues and pull requests by author, assignee, reviewer, requested reviewer and update date [27]. GitHub's GraphQL API [32] returns at most 1,000 results per search [26], and charges each query rate-limit points: the number of requests needed to fill every connection the query asks for, divided by 100, against a limit of 5,000 points an hour for a user [24].

In Linear, an issue belongs to a team, has an assignee, and has a status drawn from the team's workflow; each workflow state has a type, such as *started* or *completed*. An issue can carry *attachments*, which Linear's interface shows as links, and which this paper calls links. A link has a URL, a title, a subtitle, and key–value metadata whose "values can be any string or number"; attaching the same URL to the same issue again updates the existing link, and the `attachmentsForURL` query returns every link to a URL, with its issue [39, 42]. A personal API key acts as its owner [40].

2.2 Problem statement

A person has a GitHub account g, which can read some repositories owned by others, and a Linear user v in their own workspace. They choose *routes*: each route names some repositories and the Linear team (and, optionally, project) that their work there goes to. The task is to keep, in each route's team, a Linear issue assigned to v and linked to each GitHub item that g is part of in the route's repositories, with a status that follows the item's state on GitHub. (A pull request that fixes one of those issues may share the issue's Linear issue, Section 3.6.) Seven requirements constrain the solution:

R1 **Least privilege.** Use only the person's own credentials. Read GitHub, never write to it, and install nothing in the other organization [56].

R2 **Respect for others' work.** Change only Linear issues assigned to v, and never delete anything.

R3 **The person's edits win.** A status set by hand in Linear stays until something happens on GitHub.

R4 **Idempotence without local state.** A rerun never duplicates an issue or a link, and the program keeps no server, database or cache.

R5 **Data minimization.** Copy as little as possible from repositories that belong to someone else [19].

R6 **No surprise writes.** Show every change before making it, and stop before reading GitHub or writing anything when the setup is wrong.

R7 **Token-free and deterministic.** Run no language model, so that a run costs API requests rather than model tokens, and plan the same changes from the same state of GitHub and Linear.

Syncing in both directions, keeping descriptions or comments in sync, and syncing every issue of a repository for a whole team, which Linear's own integration does once it is installed, are not goals. Nor

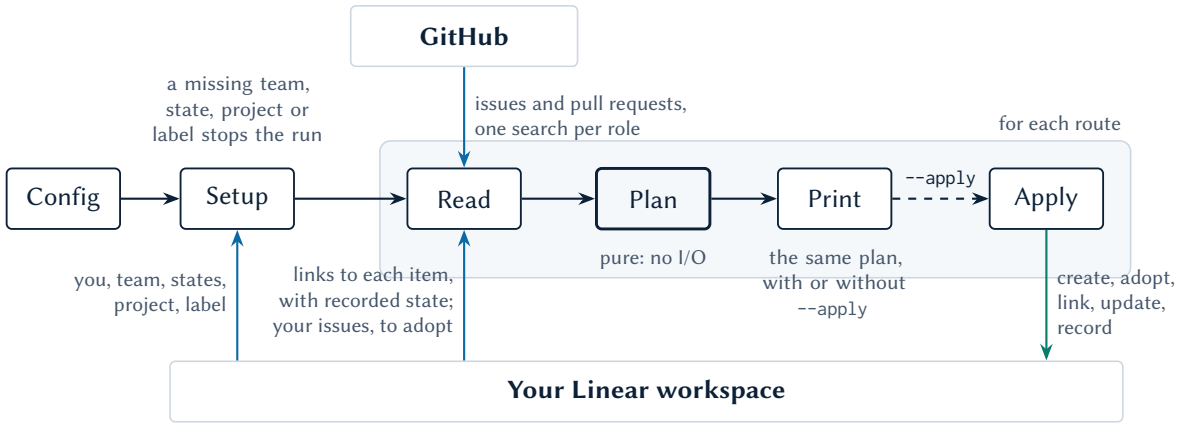


Figure 2. The steps of a run. GitHub is only read. All reads for a route happen before the plan, and all writes after it; the planner itself does no I/O. What a run writes to Linear is what the next run reads back.

is reacting instantly: webhooks would have to be created in the other organization's repositories, which needs administrator access there [28], so ferryGL polls.

3 Design

3.1 A run

A run of ferryGL has six steps (Figure 2). It loads and validates the configuration, then resolves every route's Linear setup: the API key's owner, the team, its workflow states, and the project and label if configured. A missing team, state, project or label stops the run before GitHub is read. Then, for each route, it reads GitHub and Linear, plans, prints the plan and, only when run as `ferrygl sync --apply`, makes the planned changes in Linear. Without `--apply`, a run performs every step except the writes.

3.2 What is in scope

For each route, ferryGL runs one GitHub search per enabled *role* (Table 1), restricted to the route's repositories and, if configured, to items updated since a given date. It merges the results into one item per URL that keeps every role the person has on it. Each item has a *kind*, which selects its title template and status rules: `issue`, `pr` (a pull request the person opened) or `involved` (someone else's pull request the person is part of). Commenting on an item or being mentioned in it does not bring it into scope, since that would pull in many items touched only in passing.

GitHub search has no qualifier for the author of a pull request's commits [27], so the last role is the one expensive part of a run: ferryGL reads every pull request in scope that the person did not open, with the authors of its first 100 commits, and keeps those with a commit whose author GitHub links to the person's account. Section 6.2 measures what this costs.

3.3 Homes, and state kept in the destination

Each GitHub item has at most one *home*: a Linear issue assigned to v that has a link to the item's URL. ferryGL finds homes by asking Linear for the links to each item's URL (`attachmentsForURL`) and keeping the one on an issue assigned to v. That is the whole of its lookup. It needs no mapping table of its own, so a rerun after a crash, on another machine, or months later finds the same homes, and a user may move a Linear issue to another team or project as long as the new team uses the same status names. Links on issues assigned to anyone else are ignored, so a teammate's issue that links the same pull request is never touched, and reassigning an issue hands it off: if the item is still in scope, the next run creates a new home for the user.

Table 1. Roles and the GitHub search behind each, for the login `alice`. Every search is also restricted to the route's repositories (`repo:acme/api`, or `user:acme` for every repository of an owner) and, with `since`, to `updated:>=`*date*.

Flag	Brings in	Search	Kind
`issues_assigned`	Issues assigned to you	`is:issue assignee:alice`	`issue`
`issues_created`	Issues you created	`is:issue author:alice`	`issue`
`prs_created`	Pull requests you opened	`is:pr author:alice`	`pr`
`prs_assigned`	Others' pull requests assigned to you	`is:pr assignee:alice -author:alice`	`involved`
`prs_reviewed`	Others' pull requests you reviewed	`is:pr reviewed-by:alice -author:alice`	`involved`
`review_requests`	Others' pull requests where your review is requested	`is:pr user-review-requested:alice -author:alice`	`involved`
`prs_with_commits`	Others' pull requests with your commits	`is:pr -author:alice`, then a scan	`involved`

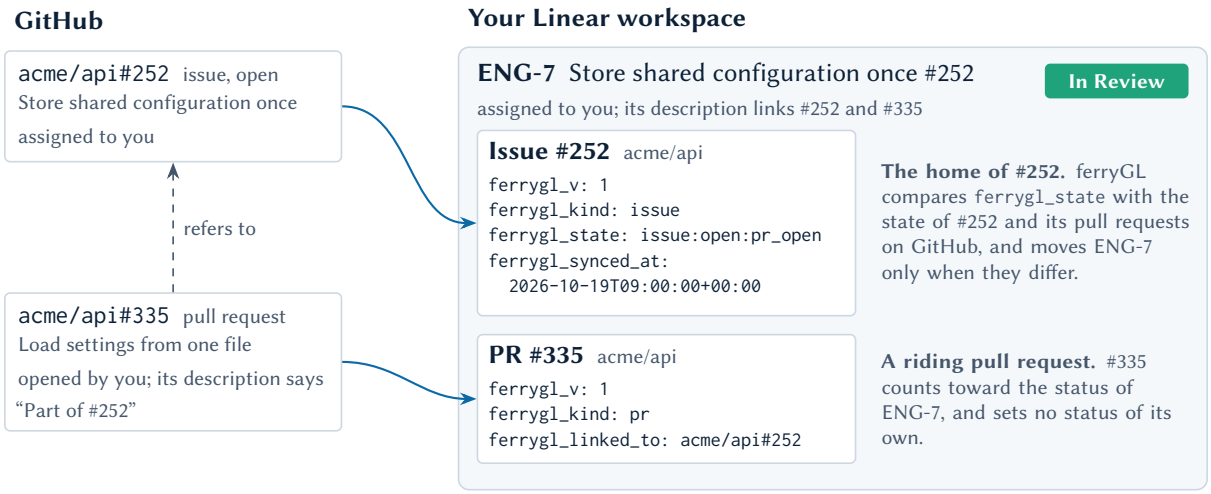


Figure 3. A home and a riding pull request. ENG-7 is the home of issue #252: its link records the GitHub state ferryGL last acted on. Pull request #335 refers to #252, so instead of a Linear issue of its own it gets a link on ENG-7, marked `ferrygl_linked_to`.

On each home's link, ferryGL records what it needs to remember as flat metadata keys, which fit Linear's string-or-number values (Figure 3): `ferrygl_state`, the GitHub state key ferryGL last acted on; `ferrygl_synced_at`, the start of the run that recorded it; `ferrygl_kind`; and the format version `ferrygl_v`. When ferryGL updates a link it replaces only its own `ferrygl_` keys and keeps any others. These keys, like the state keys below, live in users' workspaces, so they are a compatibility contract: renaming one would make every recorded link look changed.

3.4 From GitHub state to Linear status

ferryGL summarizes an item's position on GitHub as a *state key* $\sigma(i)$, one of sixteen strings such as `issue:open:pr_open` or `pr:merged` (Figure 4). For a pull request, the key follows from its kind and state and, for someone else's pull request, from the person's review roles: a pending review request maps to Todo, even after an earlier review, because it is the person's turn again. An issue closed as not planned or as a duplicate has the key `issue:closed:not_planned`. An open issue's key also depends on the pull requests linked to it (Section 3.6): it leads to In Review if one of them is open and ready for review, and to In Progress if the open ones are all drafts.

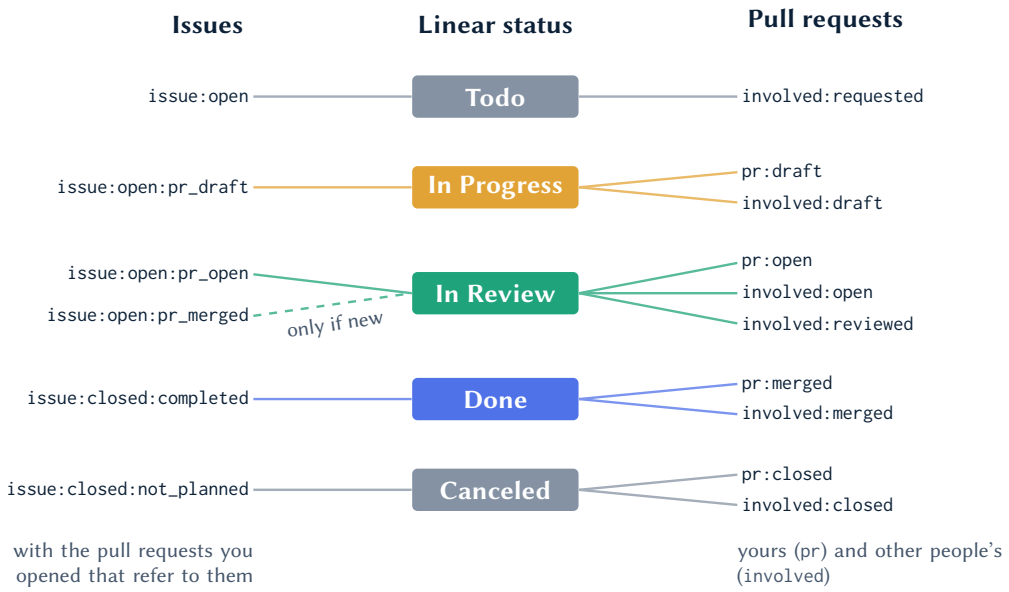


Figure 4. The sixteen state keys and the Linear status each leads to. For an issue, the first matching rule wins: closed, then a linked pull request open for review, then an open draft, then a merged one, then none. `issue:open:pr_merged` sets a status only on a new Linear issue (dashed): when an existing issue's fix merges while the GitHub issue is still open, its status stays as it is.

Each key maps to one of five status categories (Todo, In Progress, In Review, Done, Canceled), whose names in the team's workflow are configurable. The mapping has two forms: $\delta_{\text{new}}(\sigma)$ is the status of a newly created Linear issue, and $\delta_{\text{chg}}(\sigma)$ the status to move to when the state has changed. They differ only on `issue:open:pr_merged`: the fix has landed but the issue is still open, so a new Linear issue starts in In Review, while an existing one keeps its status, since the work is not done until the issue is closed.

3.5 Edge-triggered updates: the person's edits win

The central rule compares the current state key $s = \sigma(i)$ not with the Linear status, but with the state key ρ recorded on the item's home:

- If $\rho = s$, nothing happens, whatever the Linear status is.
- If $\rho \neq s$ and ρ is recorded, ferryGL moves the status to $\delta_{\text{chg}}(s)$ (an update) and records s. If the issue already has that status, or $\delta_{\text{chg}}(s)$ keeps the status, it only records s (a record).
- If no state is recorded ($\rho = \bot$), this is the link's *first sync*: a link the user added by hand, or one ferryGL just added to an issue the user made. ferryGL then moves the status only to close the issue, to Done or Canceled, when GitHub says the work is done or canceled and the Linear issue is still open. It never reopens an issue, or moves it in any other way, on a first sync.

A status therefore moves only on an *edge* in the GitHub state, relative to a memory that lives in the destination (Figure 5). A level-triggered sync, which computes the desired status from GitHub on every run, would overwrite the person's edits each time. Level-triggered reconciliation is the right model when the source of truth should win continuously, as in the controllers of Borg, Omega and Kubernetes [10]. Here two parties own different things: GitHub owns what happened on GitHub, and the person owns their own tracker. Recording the last synchronized state and comparing against it is how file synchronizers tell which replica changed [7, 52]; ferryGL applies the same update detection to the source only, and treats the Linear status as the person's between edges. Algorithm 1 gives the rule in full, with the cases of Section 3.6.

Two properties follow, and Section 6.1 tests both.

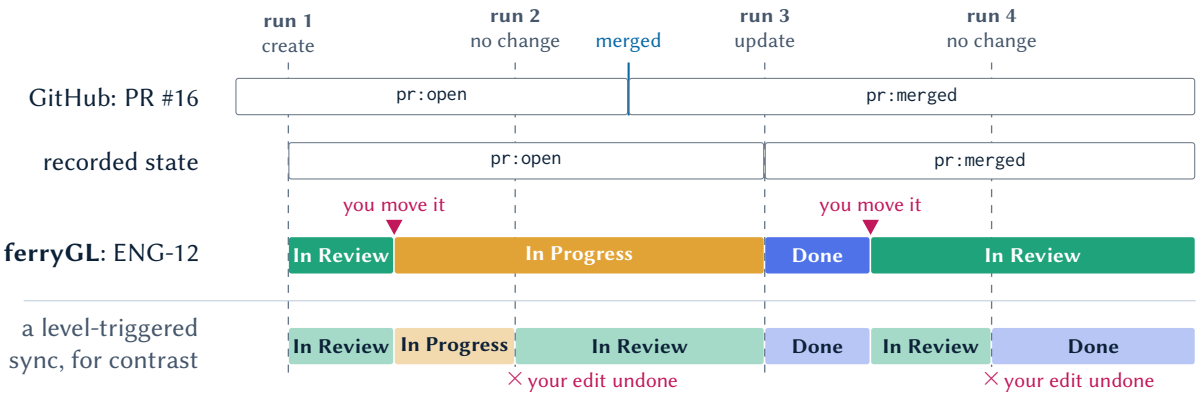


Figure 5. Edge-triggered updates for one pull request and its Linear issue. Run 1 creates ENG-12 in In Review and records `pr:open`. The user moves it to In Progress; run 2 sees that the recorded state still matches GitHub and changes nothing. Run 3 sees the merge and moves it to Done. The user then reopens it, and run 4 leaves it alone. A level-triggered sync (bottom) would undo both edits.

Edits persist. If a run completes its writes, every home it touched records the state key it acted on. If, by the next run, the key ferryGL computes for an item is unchanged, the first branch applies and the home's status is left alone, whatever the user did to it in between. (The key of an issue depends on its linked pull requests, so a new review request or a merge is an edge; Section 6.1 shows one case where the key changes although, by the documented rules, it should not.)

Quiescence. If neither GitHub nor Linear changes after a run that completed its writes, the next run plans no writes: every item in scope has a home that records its current key, and every rider is linked. Reruns are therefore safe to schedule as often as wanted.

3.6 Pull requests that ride with an issue, and adoption

An issue and the pull request that fixes it are one piece of work. When a pull request the person opened refers to one of their synced issues in the same route, it *rides* with the issue: instead of a Linear issue of its own, it gets a link on the issue's home, with `ferrygl_linked_to` naming the issue (Figure 3). A pull request refers to an issue when GitHub lists the issue as one it will close, or when its title or description mentions it as `#252`, `acme/api#252` or by URL. The issue's state key accounts for its linked pull requests, so the home follows the issue and its fix together, and the rider's link sets no status of its own: each Linear issue has one status source. The edge cases follow from that. A pull request that had a home of its own before it referred to the issue keeps it, and still counts toward the issue's status; an issue with no home *joins* its pull request's home if the pull request refers to no other synced issue. A rider's link stays, and keeps counting, when the reference is later edited out, and a rider never moves its issue's home on its own, even when the issue leaves the person's scope. Someone else's pull request always gets its own home, and a pull request that refers to several of the person's issues rides with each of them.

People often create a Linear issue for a GitHub item themselves before they install ferryGL. Before creating a new issue, ferryGL therefore looks for one of the person's issues in the route's team that already mentions the item: its URL in the description or in a link, or, for a route with a single repository, a title ending in *#number* with no link to any other GitHub item. A candidate already linked to a different GitHub item, or already home to another item, never matches. Exactly one match is *adopted*: ferryGL adds its link and applies the first-sync rule, so the adopted issue's status moves only to close it. Several matches are *skipped*, with their identifiers: the person adds the link to the right one, and the next run picks it up.

Algorithm 1 Planning one item i, with its linked pull requests L and riders R (Section 3.6). $A(u)$ are the links to URL u returned by Linear, C the user's issues in the route's team, and v the user.

$s \leftarrow \sigma(i, L)$
$h \leftarrow$ the first link in $A(url(i))$ on an issue assigned to v, if any ▷ the home
if h exists **then**
    $\rho \leftarrow$ the state recorded on h, or $\perp$
    if $\rho \neq s$ **then**
        if $\rho = \perp$ **then** ▷ first sync: only close
            $c \leftarrow \delta_{\text{chg}}(s)$ if it is Done or Canceled and h's issue is open, else none
        else
            $c \leftarrow \delta_{\text{chg}}(s)$ ▷ none for `issue:open:pr_merged`
        emit update(h, c) if c is a status other than the issue's, else record(h); both record s on h
    emit link(r) for each $r \in R$ not yet linked to h's issue
else
    $M \leftarrow \{\, m \in C$ unclaimed, mentioning i or a rider, and linked to no other GitHub item $\}$
    if $|M| > 1$ **then**
        emit skip(i, M) ▷ the user decides
    else if $|M| = 1$ **then**
        emit adopt(i, m) under the first-sync rule, and claim m
    else
        emit create(i) with status $\delta_{\text{new}}(s)$, linking i and R

3.7 Writes, failures and retries

Each change is applied in a fixed order: the status first, then ferryGL's comment (if comments are on), then the links. The recorded state therefore changes last: if the status update or the comment fails, the old state stays recorded, and the next run plans the same change again. Each change is applied independently, so one failure does not stop the others; the run then exits with status 1.

Retries follow the idempotency of each request [20, 33]. A request gets up to three attempts, after waits that honor `Retry-After` up to 60 seconds and are otherwise 2 and then 4 seconds, and one that takes over 30 seconds counts as a network error. Rate-limited responses (HTTP 429, or GitHub's 403 with `Retry-After`) are always retried. Network errors, timeouts and HTTP 502, 503 and 504 are retried only for idempotent requests: every read, a status update, and a link create or update, which Linear makes idempotent per issue and URL [39]. Creating an issue or a comment is not idempotent, since Linear may have created it before the connection failed, so ferryGL reports the error instead of retrying. If the issue was created, the next run finds it through the GitHub URL in its description and adopts it, so no duplicate is created; a comment, however, may be posted twice (Section 8).

4 Implementation and safety

ferryGL 0.2.1 is a Python package of about 1,900 lines in eight modules, with one runtime dependency, the HTTP client HTTPX [18]. Its structure follows the "functional core, imperative shell" pattern [8, 48]: `plan.py` is a pure function from the GitHub items, the Linear links and candidates, and the configuration to a list of changes, and everything that talks to the network lives around it. Its 155 tests run in under a second: each planning test states a scenario in plain terms, the clients are tested against `httpx.MockTransport`, and the commands run end to end against fakes.

Configuration and commands. The configuration is a TOML file [53] in which only the routes are required. Unknown keys are errors, so a misspelled setting cannot be silently ignored, and a published JSON Schema [63] gives editors completion and inline checks. A complete configuration can be as short as:

Table 2. The eight invariants, how the design enforces them, and the evidence. Property names in italics refer to Table 3.

Invariant	Mechanism	Evidence
I1. GitHub is read-only	The GitHub client only sends GraphQL queries	No mutation among all requests (Section 6.2)
I2. Only the user's Linear issues; nothing deleted or rewritten	Writes target homes or new issues assigned to the user; the Linear client has no delete, and its update sets only the status	*ownership*
I3. The user's edits win	Edge-triggered updates against the recorded state; first syncs only close	*status source*, *edits win*
I4. One home per item, one status source per Linear issue	Lookup by link; claimed candidates; riders set no status	*one home*, *coverage*
I5. Idempotent reruns	Homes found by link; only idempotent requests retried	*quiescent*, *recovery*
I6. A pure planner	All reads before planning, all writes after; the planner does no I/O	By construction; *order independent* found one order-dependent case (Section 6.1)
I7. Copy as little as possible	Only titles, numbers, repositories, URLs, states and roles by default	*minimal data*
I8. No surprise writes	Preview by default; setup checked before reading GitHub	*dry run*

```
[[route]]
repos = ["acme/api", "acme/web"]
team = "ENG"
```

`ferrygl check` confirms that the Linear key works, that the GitHub token can see every repository, and that each route's team, states, project and label exist; it writes nothing. `ferrygl sync` prints the plan, one line per change, and `ferrygl sync --apply` prints the same plan and makes it:

```
acme/api#252  issue     update  ENG-7 Todo -> In Review
acme/api#335  pr        link    -> ENG-7
acme/api#360  issue     create  "Cache the session lookup #360" -> Todo
acme/api#348  involved  adopt   -> ENG-9
4 changes (1 create, 1 adopt, 1 link, 1 update). Run with --apply to make them.
```

The exit status is 0 when everything worked, 1 when a route could not be read or a write failed, 2 for a setup or configuration error (in which case nothing was read from GitHub or written to Linear), and 130 when interrupted. With comments turned on, every change also posts a Linear comment that starts with the marker `(BY-FGL)`, so people can tell ferryGL's changes from their own.

Invariants. ferryGL usually runs in repositories that belong to someone else, often private ones whose contents are covered by an agreement, so its safety rules matter more than its features. The repository's contributor guide states them as eight invariants, which human contributors and coding agents alike must not break without the project owner's decision. Table 2 lists each one with the mechanism that enforces it and the evidence from Section 6. Two hold by construction rather than by care. The GitHub client has no code path that sends a mutation. The Linear client exposes exactly five mutations: `issueCreate`, `issueUpdate` with only a state, `attachmentCreate`, `attachmentUpdate` and `commentCreate`. No method can delete an issue, a link or a comment, or edit a title, a description or a comment, so no bug in the planner can make ferryGL do so.

Credentials and data. ferryGL sends the GitHub token only to `api.github.com` and the Linear key only to `api.linear.app`, and stores neither. A read-only GitHub token is enough. The Linear key acts as the user, so ferryGL's own rules, not the key's scope, keep it to the user's issues. By default, ferryGL writes to Linear only the GitHub title and number (the title template can leave the title out), the repository, the item's URL, its state key, and the user's role as implied by the kind. It reads descriptions to find the issues a pull request refers to, but copies one only if the user turns that on, once, when the issue is created. It never copies GitHub comments, reviews, labels, code, or other people's names or logins. Its own comments name only GitHub items, states and Linear status names, and it keeps nothing on the user's machine.

5 Token-free automation for AI agents

An AI agent asked to keep Linear in step with GitHub can do the job in two ways. It can do the job itself: search GitHub for each role, look up each item in Linear, apply the rules of Section 3, and make the writes, reading every item in scope on every run. Or it can delegate the job to a token-free tool: run `ferrygl sync`, read the plan, run `ferrygl sync --apply`, and report. Its tokens then go to what ferryGL prints, a line per change, or `No changes.` on a quiet run, instead of to every item in scope. Sections 6.4 and 6.5 measure the difference.

An interface agents can use. ferryGL needs nothing agent-specific: any agent that can run a command can use it, through the same interface a person uses. Three of its properties make that safe and cheap. A run previews by default, so an agent can show the plan to its user, or check it, before it applies it. The output is compact and regular: one line per change, with the item, its kind, the action and its details, then a one-line summary, and every skip line says what decision it needs. And the exit status says what happened. ferryGL's documentation has a page for agents, and its website publishes an `llms.txt` index of the documentation for language models [35]. Instructions as short as these are enough:

```
To sync my GitHub work into Linear, run `ferrygl sync` and show me the plan.
Run `ferrygl sync --apply` only after I approve it. If a line says skip,
tell me what ferryGL needs. If the exit status is 2, report the error and stop.
Never change GitHub or Linear yourself to do this.
```

Least privilege for agents. Delegating can also narrow what an agent is able to do. An agent whose only access to GitHub and Linear is ferryGL cannot write to GitHub, change a teammate's Linear issue, or delete anything, whatever it is asked or gets wrong, because the invariants of Table 2 hold for every caller [56]. That guarantee holds only if the agent's shell is limited to the `ferrygl` commands and the credentials are kept out of its reach: an agent with an unrestricted shell and the keys in its environment could call either API directly.

A general pattern. The same split applies wherever an agent is asked to do routine, rule-based work again and again: keeping trackers in step, triage by fixed rules, recurring reports. Once a task's rules are known, writing them into a program moves the task's marginal cost from inference to API requests, and makes its behavior repeatable. The program is written once, tested and shared; the agent decides when to call it and reads what it reports (Section 7 places this among other ways to make agents cheaper).

6 Evaluation

The evaluation asks four questions. Do the safety invariants hold over long, random histories in which GitHub changes, the user edits Linear, teammates link the same items, and writes fail (Section 6.1)? What

Table 3. Properties checked by the simulation, how many times each was checked, and how many checks failed. A check is one evaluation of the property on one run's state, writes or plan.

Property	Statement	Checks	Failed
dry run	A run without `--apply` writes nothing	24,000	0
quiescent	A rerun with nothing new on GitHub or in Linear writes nothing	24,000	0
one home	Each GitHub item is linked as a home from at most one of the user's issues	80,923	0
ownership	Every write targets an issue assigned to the user	49,178	0
status source	A status moves only with a new recorded state, or to close on a first sync	4,742	0
edits win	After a step that changed only Linear, every status the user set is still set, unless a first sync closed the issue	4,098	0
coverage	Every item in scope is linked from one of the user's issues, unless it, or the issue it rides with, was skipped	86,168	0
consistent	A pull request's home records its current state and, unless the user moved it, has the status Figure 4 gives	75,204	0
foreign metadata	Metadata keys that are not ferryGL's survive every update	11,302	0
minimal data	No write carries a description or another user's login, and no comment a title	49,178	0
recovery	After failed writes, one clean run succeeds and converges	4,278	0
order independent	The plan does not depend on the order of Linear's lookup results	14,065	29

does a run cost in GitHub requests and rate-limit points, and in planning time (Sections 6.2 and 6.3)? What would an AI agent spend to do the same job itself, or by delegating it to ferryGL (Section 6.4)? And how does that vary across models and providers (Section 6.5)? All of the data is synthetic.

6.1 Randomized, model-based simulation

Method. The simulation runs ferryGL's own reading, planning and writing code, unchanged, against two in-memory stand-ins with the same methods as its GitHub and Linear clients. The GitHub stand-in answers each role's search from a model of issues and pull requests, in the shape of GitHub's GraphQL nodes, so ferryGL's result parsing and reference detection run too. The Linear stand-in keeps issues, links and comments, and follows Linear's documented behavior: links are returned by URL across the workspace, up to 20 per URL, and attaching a URL an issue already has updates that link. As in property-based testing [15], each scenario is generated at random from a seed: one or two repositories, a random configuration (each role on with probability at least 0.85, comments on or off), and a few initial items. Then, for 12 steps, random events happen and ferryGL runs. GitHub events open and close issues, open, ready, draft, merge, close and reopen pull requests, request, give and withdraw reviews, assign and unassign the user, add and remove references, and push commits. Linear events have the user move statuses, hand issues to a teammate, link items by hand and write issues that mention items, and have a teammate link the same items and the user add their own metadata to ferryGL's links. Three in ten steps change only Linear. Each step runs ferryGL up to four times: a preview; an apply in which 5 % of writes fail, half of them *lost* (they reach Linear but lose their response) and half *rejected*; a clean recovery run if any write failed; and a final clean rerun.

After every run, the simulation checks the properties of Table 3 against the stand-ins' state and the writes ferryGL sent. GitHub descriptions in the model carry a secret marker and mention other users, so *minimal data* can check that no write carries either, and that no comment carries a GitHub title; description copying, which is off by default, stays off, so this checks the default configuration. *Order independent* is a metamorphic relation [13, 14]: planning the same inputs with every list of Linear lookup results reversed must give the same changes, since Linear promises no particular order. Reversal is a single permutation, so the check can expose order dependence, but cannot show its absence.

Results. 2,000 scenarios produced 55,538 events, 74,139 runs and 49,178 writes, of which 2,250 failed. The plans included 11,421 creates, 5,734 updates, 1,598 links, 821 records, 412 adoptions and 133 skips, so every action was exercised. The whole simulation takes about ten seconds. Every safety property held in every one of its 413,071 checks (Table 3). Write failures left 776 Linear issues created without their links; the next clean run adopted all 776 of them through the URL in their description, and every recovery converged in one run.

A finding: order dependence. The metamorphic check failed in 29 of 14,065 non-trivial checks, all with the same cause. A pull request that refers to two of the user's issues rides with both, so its URL is linked from both of their Linear issues. If one reference is later edited out, the planner looks up the pull request's links and takes the first one on one of the user's issues as the pull request's own. When that first link is on the issue it still refers to, the pull request rides with that issue only and stops counting toward the other issue's status: that issue's state key changes, say from `issue:open:pr_open` to `issue:open`, and its Linear issue moves back to Todo. When the first link is on the other issue, the rule of Section 3.6 applies as documented (the link stays and keeps counting), and nothing moves. Linear promises no order for these results, so the outcome is not determined. The planner itself is still pure; what fails is its invariance to the order of an input that is semantically a set, and with it requirement R7 and the documented rule for riders. The case is narrow (a pull request that refers to several of the user's issues and then drops one reference) and moves only the status of the user's own issue. A fix is to choose among a pull request's links deterministically, preferring a link on the home of an issue it still refers to, and to keep counting the pull request toward the status of every issue whose home has its link.

6.2 Request cost

Method. The cost experiment runs ferryGL's real GitHub client against a mock of GitHub's GraphQL endpoint, built on `httpx.MockTransport`, that pages results as GitHub does and stops at 1,000 results per search. The repository holds a fixed set of the user's own items (12 issues, 8 pull requests) and a growing number of other people's pull requests; one in ten requests the user's review, one in ten was reviewed by them, one in twenty is assigned to them, and one in twenty has one of their commits. Per route, ferryGL sends one query for the token's login and, for each enabled role with n results, $\max(1, \lceil n/50 \rceil)$ search pages, or $\max(1, \lceil \min(N, 1000)/25 \rceil)$ pages for the commit scan over N pull requests the user did not open. GitHub's cost formula [24] charges by the sizes a query asks for, not the results it returns. A search page asks for 50 items with up to 20 closing references each, which needs 51 requests to fill and costs 1 point. A commit-scan page asks for 25 pull requests, each with 20 closing references and 100 commits of up to 5 authors, which needs $1 + 25 + 25 + 25 \times 100 = 2{,}551$ requests and costs 26 points.

Results. At every size, the requests ferryGL sent matched this model, and none was a mutation. Figure 6 shows the result. Without the commit scan, a route with 1,000 other pull requests in scope costs 9 requests and 9 points. With it, the same route costs 49 requests and 1,049 points, about a fifth of a user's hourly 5,000; the scan accounts for 40 of the requests and 1,040 of the points. Beyond 1,000 pull requests, GitHub's limit on search results truncates the scan: the cost stops growing, but pull requests past the 1,000th are no longer checked, so some of the user's commits go unseen. This is why ferryGL's documentation recommends setting `since`, which bounds every search by update date, and turning `prs_with_commits` off when the role is not needed.

6.3 Planner time

A benchmark times the planner alone, on synthetic routes of 10 to 10,000 items, four in ten of them issues, three in ten the user's pull requests (most referring to an issue) and three in ten other people's, as the median of seven runs on an AMD Ryzen 9 9950X3D with CPython 3.12. When every item has a home, one in ten with an out-of-date recorded state, planning time grows linearly: 0.38 ms for 100 items

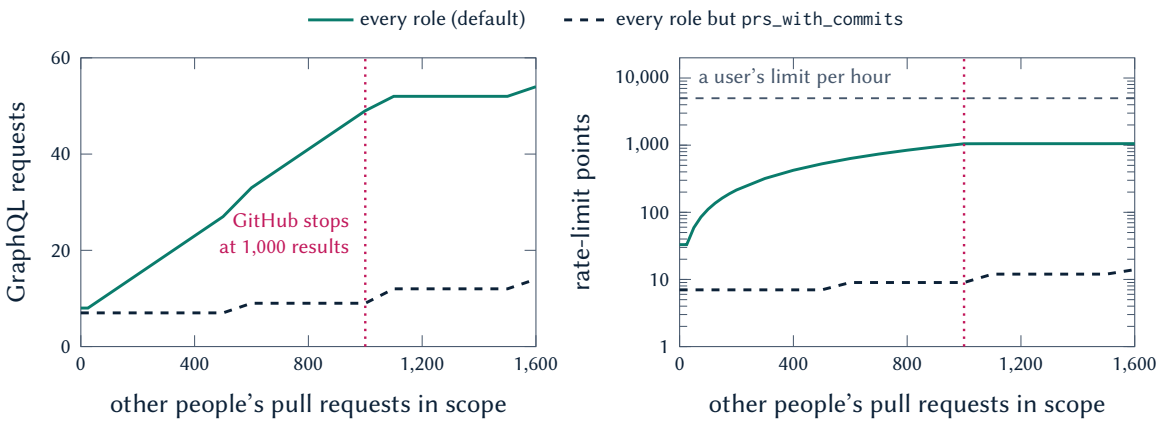


Figure 6. GitHub requests (left) and rate-limit points (right, log scale) for one route, as the number of other people's pull requests in scope grows. The commit scan (`prs_with_commits`) dominates the cost. GitHub returns at most 1,000 results per search, so past that point the cost flattens and the scan misses pull requests.

and 46 ms for 10,000. When a fifth of the items are new, the planner also matches each new item against each of the user's issues for adoption, so its time grows faster than linearly, to 288 ms for 10,000 items, far more items than one person is usually part of. Either way, a run's time is dominated by its network requests.

6.4 What an AI agent would spend, and what delegating to ferryGL saves

Method. No agent was run for this paper. Instead, the experiment measures the work each strategy of Section 5 implies, and prices it. It runs ferryGL through stand-ins that record every call, over 48,000 runs of random histories generated as in Section 6.1, without injected failures, and over single routes of 10 to 800 items, with the commit scan off, in which a twentieth of the items change on GitHub between runs. For an agent doing the job itself, the stand-ins record the size of each result it would have to read and of the arguments of each call it would have to send, if its tools returned exactly what ferryGL's queries return. For a delegating agent, they record what ferryGL prints, which is all it reads. A cost model then prices each agent loop:

- An agent doing the job itself starts each run with ferryGL's rules as instructions (its sync rules and link-metadata convention, about 9,700 characters), nine tool definitions shaped after ferryGL's reads and writes (about 2,600 characters), the 286-token tool-use system prompt [4], and a short request to sync: about 3,361 tokens. A run has three dependent steps (read GitHub and the Linear setup; look up the links to each item; write) and a closing turn, and a step takes one turn per batch of parallel tool calls. Letting all of a step's writes share a turn favors the agent, since a link can only be added once its issue exists.
- A delegating agent starts with a five-sentence instruction, which, unlike the one in Section 5, lets it apply the plan without asking, and the bash tool, whose definition adds 325 tokens to the tool-use system prompt [4]: about 735 tokens. It runs `ferrygl sync`, reads the plan, runs `ferrygl sync --apply`, reads the result, and reports, in three turns.
- Prompt caching is on: each turn reads the previous prompt from the cache and writes only what is new. Scheduled runs are hours apart, so each run starts with a cold cache.
- Prices are Anthropic's published rates for Claude Opus 5.5, Sonnet 5.5 and Haiku 5.5, including Haiku 5.5's fivefold rates for prompts over 100,000 tokens [4].

Tokens per run counts every token the model processes: the input of every turn, including the cached prefix each time it is read, plus the output. A cached token costs a tenth or less of a fresh one, so dollars, not tokens, measure the saving. Two sets of assumptions bracket how an agent behaves (Table 4), and

Table 4. The two sets of assumptions about how an AI agent works. Neither describes a measured agent; together they bracket one. Characters per token are measured (Table 8): over its seven distinct tokenizers, the most characters per token for lean and the median for plausible, for each kind of text.

Assumption	Lean	Plausible
Average length of a GitHub description it reads	0 characters	500 characters
Tool calls per turn	all of a step's	at most 5
Reasoning tokens per turn, billed as output	100	500
Characters per token: JSON / prose, for models without a public tokenizer	2.93 / 4.06	2.81 / 4.02
Shared by both	tools that return only the fields ferryGL reads; ferryGL's rules as instructions; prompt caching, cold at the start of each run; each provider's published prices; a model's own tokenizer where it is public	

both strategies are priced under both. The *lean* assumptions give the agent every advantage, so they approach the least an agent could spend. The *plausible* ones are a middle estimate, not a worst case: general-purpose GitHub and Linear tools return fuller records than ferryGL reads, so an agent using them would read more.

Results. On a route of 100 items, an agent doing the job itself would process 91,620 to 217,857 tokens per run; an agent delegating it to ferryGL, 3,364 to 5,769; and ferryGL on its own, none (Table 5, Figure 7). On Claude Opus 5.5, doing the job itself costs $10.91 to $18.48 a month at two runs a workday, and delegating it $0.71 to $1.95; hourly runs cost four times as much. Four findings stand out.

- **Delegating cuts the cost many times over, more the larger the route.** A delegating agent costs 3 to 4 times less than one doing the job itself at 10 items, 9 to 15 times less at 100 (with 27 to 38 times fewer tokens), and 77 to 79 times less at 800. After the first run, its cost grows only slowly with the route, because ferryGL prints a line per change, not per item: 3,364 to 5,769 tokens at 100 items, and 4,991 to 7,414 at 800. What remains is mostly its own instructions and reasoning.
- **A run that finds nothing new costs almost as much.** An agent that reads the whole scope learns that nothing changed only after reading it. On the same route, a run that finds nothing new reads fewer tokens (54,966 to 106,048) but still costs $0.22 to $0.36 on Opus 5.5, against $0.25 to $0.42 for a run that finds 4 changes. In the random histories, each applying run is followed by a rerun that finds nothing new, and those reruns account for 46 % of the agent's spend.
- **An agent's own cost grows faster than its scope.** Its context grows with every item read, and is read again on every turn. At 400 items, the plausible agent processes 1,941,133 tokens per run, $73.01 a month on Opus 5.5 at two runs a workday. Haiku 5.5's long-prompt rates show as a jump once prompts pass 100,000 tokens, past 100 items under the plausible assumptions and past 200 under the lean ones. The longest prompt, 35,955 to 55,454 tokens at 100 items, reaches 257,464 to 397,407 at 800, which only models with long context windows can hold. The first run, which creates every Linear issue, costs $0.69 to $1.70 on Opus 5.5 for 100 items.
- **A smaller model costs less, at a risk not measured here.** Sonnet 5.5 halves the cost of Opus 5.5, and Haiku 5.5 cuts it by more than an order of magnitude at modest scope. Whether a smaller model applies ferryGL's rules reliably is exactly what this estimate cannot say; a delegating agent needs only to run two commands and read their output.

Interpretation. For one person with a modest route, an agent's bill is small in absolute terms: $5.46 to $18.48 a month on Sonnet or Opus at two runs a workday, and under a dollar on Haiku. The case for

Table 5. What an AI agent would spend on a route of 100 items, for a run that finds 4 changes to make, doing ferryGL's job itself or delegating it to ferryGL. Monthly costs assume two runs a workday. Tokens per run, which include cached reads, do not depend on the Claude model. Estimated with the cost model of Section 6.4.

		Doing the job itself		Delegating to ferryGL	
Model	Assumptions	Tokens per run	Per month	Tokens per run	Per month
Claude Opus 5.5	lean	91,620	$10.91	3,364	$0.71
	plausible	217,857	$18.48	5,769	$1.95
Claude Sonnet 5.5	lean	91,620	$5.46	3,364	$0.36
	plausible	217,857	$9.24	5,769	$0.97
Claude Haiku 5.5	lean	91,620	$0.28	3,364	$0.018
	plausible	217,857	$0.50	5,769	$0.049
ferryGL alone		0 tokens, $0			

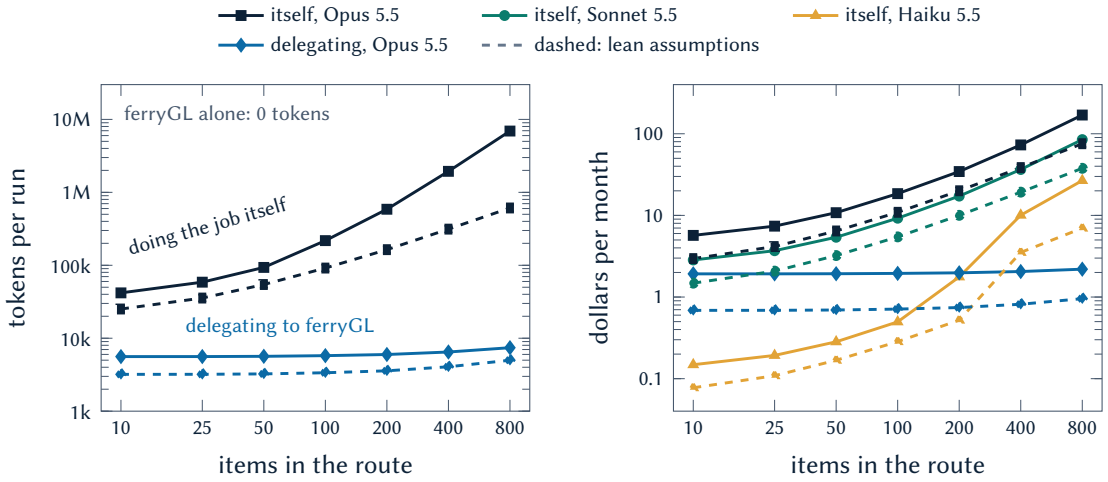


Figure 7. What an AI agent would spend, by the size of the route, doing ferryGL's job itself or delegating it to ferryGL: tokens per run (left), which do not depend on the Claude model, and dollars per month at two runs a workday (right), when a twentieth of the items change on GitHub between runs. Solid lines use the plausible assumptions, dashed lines the lean ones. Both axes are logarithmic. ferryGL alone spends no tokens.

token-free automation rests less on the size of the bill than on its shape. An agent's own cost recurs on every run, including the runs that find nothing to do; it grows with the route and the schedule; and it multiplies with every person who runs it. It also buys no guarantees: the estimate assumes the agent applies the rules correctly every time, which language models do not promise [6, 51], and a run that gets one wrong (a status moved over the user's edit, a duplicate issue, a teammate's issue changed) costs more than its tokens. Delegating shows that the two are not alternatives: an agent that runs ferryGL keeps its judgment for the decisions that need it, such as when to run and what to do about a skipped item, spends a few thousand tokens per run on them, and inherits ferryGL's guarantees.

Limits of the estimate. This is a model of agents, not a measurement of them. Reasoning tokens per turn, calls per turn and turns per run are assumptions, chosen in the lean case to favor the agent. Every provider is given Anthropic's tool-use overhead (286 tokens, and 325 more for the bash tool), since not all publish theirs, and Claude's tokenizer is not public, so measured ratios stand in for it. Both strategies read the whole scope on every run, because ferryGL does: an agent that kept a watermark and searched only for items updated since its last run would read far less on quiet runs, at the price of keeping state of its own, and ferryGL could do the same. The model does not represent context windows: past a few hundred items, an agent on a model with a shorter window would have to split a run or compact

its context. Finally, the agents' correctness, determinism and latency are not measured. Running real agents with these tools over the same histories would replace the assumptions with measurements, and would also count how often an agent doing the job itself breaks the invariants of Table 2.

6.5 Across 24 models from 10 providers

Method. This experiment repeats the estimate on 24 current models from 10 providers: for each provider, its flagship and at least one cheaper model, among them the leading open-weight models as served by their makers (Table 7). Everything but the model is held fixed: the measured workload (one user's route of 100 items, on a run that finds 4 changes to make, with 21 requests to GitHub and Linear), the three strategies, and the lean and plausible assumptions. Three inputs vary by model, each from a published source read on 19 October 2026.

- **Prices.** Each provider's standard API prices for input, cache writes, cache hits and output, under its own caching rules: whether a cache write costs more than input, the shortest prompt it caches, and any higher price for long prompts [1, 3, 4, 17, 29, 45–47, 50, 64, 69]. Where a price is set to rise on an announced date, the later price is used, since the comparison looks a year ahead; where some hours cost less, the full price is used.
- **Tokenization.** The open-weight models' own tokenizers, run on the text of the workload (Table 8). JSON tool results, most of what an agent reads, come to only 2.4 to 2.9 characters per token, well below the four characters per token often assumed for prose. Claude, GPT-6, Gemini, Grok, Mistral Large 4 and MiniMax-M3.1-Flash have no public tokenizer, and take the measured median (plausible) or maximum (lean), per kind of text; the measured set includes OpenAI's published `o200k_base` as one proprietary tokenizer.
- **Speed.** Each model's output speed and time to first answer token on its maker's API, as measured by Artificial Analysis with a 10,000-token prompt, in the fastest configuration it measured [5]. A turn takes that time, which includes the model's reasoning, plus its visible output at that speed, plus its requests, which run in parallel within a turn. ferryGL's requests run one after another, at an assumed 0.5 seconds each. No speed was measured for 3 models, whose times are left out.

The schedule and team size are the same for every model: two runs a workday (44 a month) or hourly over an eight-hour workday (176 a month), for one user and for teams of 10, 50 and 100, each user with a route of their own. ferryGL on its own spends no tokens. On a user's machine it costs nothing; in GitHub Actions, each run is a job billed as one minute, at $0.006 a minute for a private repository [23], and a user's 44 runs a month fall well within the 2,000 minutes GitHub Free includes [22]. The comparison counts every minute as paid, as an upper bound.

Results. Figure 8 shows the results, and Table 7 gives them per model; the data have every combination of model, strategy, assumptions, schedule and team size.

- **At team scale, an agent doing the job itself costs from hundreds to tens of thousands of dollars a year.** For a team of 100 at two runs a workday, it costs $596 to $59,576 a year, depending on the model, under the plausible assumptions, and $341 to $34,126 under the lean ones, processing 11.2 to 12.7 billion tokens a year (plausible). Hourly runs cost four times as much.
- **The choice of model spans two orders of magnitude.** The dearest flagship costs 100 times as much as the cheapest small model, and within one provider the flagship costs 1.9 to 100 times as much as the cheapest model.
- **Delegating to ferryGL cuts every model's bill** 7 to 16 times under the plausible assumptions (7 to 21 times across both sets), to $59 to $6,249 a year for a team of 100, because the agent reads ferryGL's plan instead of every item.
- **ferryGL alone spends nothing on inference**: $0 on users' own machines or within GitHub

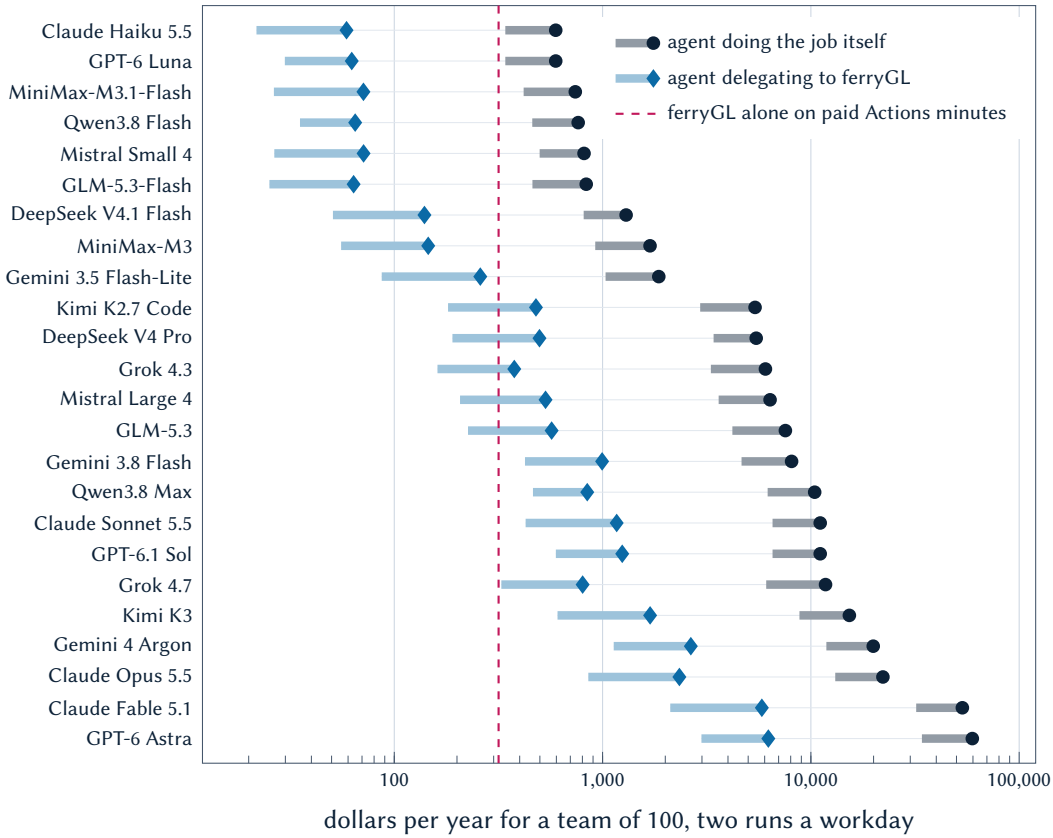


Figure 8. Dollars per year for a team of 100 at two runs a workday, by model, from the cheapest to the most expensive agent doing the job itself. Each bar spans the lean to the plausible assumptions, and its mark is the plausible estimate; the axis is logarithmic. Delegating to ferryGL cuts every model's bill; ferryGL alone spends nothing on inference, and at most $317 a year on paid GitHub Actions minutes. Table 7 gives the values.

Actions' free minutes, and at most $317 a year for a team of 100 on paid minutes. That is less than an agent doing the job itself costs on any model, even under the lean assumptions. A delegating agent costs less than that on the 9 cheapest models, but it still needs ferryGL to run somewhere, so its cost comes on top.

- **ferryGL alone is also the fastest**, by this estimate. A run takes about 11 seconds, nearly all of it its 21 requests at the assumed half second each. An agent doing the job itself takes 25 to 415 seconds, although its requests run in parallel, and a delegating agent, which runs ferryGL twice, 21 to 191 seconds. Models measured only with long reasoning take minutes.

Caveats. Prices change often, and several here are introductory, set by the time of day, or for a preview (notes a to c of Table 7); the script reruns the comparison from a table of prices. Speeds were measured by a third party, on prompts that ask for longer answers than an agent's tool calls. A model measured only with reasoning on is likely slower here than it would be on short tool calls; a model measured without reasoning is timed in a faster configuration than the one whose reasoning tokens are priced, which favors it. Open-weight models are counted with their published tokenizer, which the hosted model may not match exactly. Flat-fee subscription plans, and batch discounts, which do not suit an interactive loop, are not modeled.

6.6 Comparison with the alternatives

Table 6 compares ferryGL with the ways people bridge the gap today. The automated options differ mainly in what they need from the other organization and in what each run costs. Linear's integration

Table 6. Ways to track work in another organization's GitHub repositories in your own Linear workspace.

	Linear's GitHub integration	Copying by hand	An AI agent	ferryGL
Needs the other organization's owners or admins	Yes, to install it	No	No	No
Credentials	A GitHub App in their organization	Yours	Yours, given to the agent	Yours: a read-only GitHub token and a Linear API key
Scope	Linked repositories, for the whole team	What you copy	What you ask for	Items you are part of, for you
Cost of each run	None	Your time	Model tokens: 91,620 to 217,857 per run for 100 items (Section 6.4)	API requests; no model tokens. An agent that runs it: 3,364 to 5,769 tokens per run
Same result every time	Yes	No	Not guaranteed	Yes
Preview before writing	n/a: it acts on GitHub events	n/a	Depends on the agent	Yes, by default

is the right tool when it can be installed: it is maintained by Linear, covers every issue of a repository for the whole team, and can even sync issues both ways [41]. General workflow-automation services offer GitHub and Linear connectors too [43, 49, 70], but the user would have to build the roles, homes, riding and edit-preserving rules as workflows, and some services need an organization owner or administrator to connect a GitHub organization [61].

6.7 Threats to validity

The simulation's conclusions are only as good as its stand-ins. They follow Linear's and GitHub's documented behavior, but real services differ in details the stand-ins leave out, such as the order of results (which the metamorphic check varies), eventual consistency between a write and the next read, and rate limiting. Reads and writes against a real Linear workspace had not yet been exercised at the time of writing; GitHub reads have been exercised against a real repository. The random histories follow hand-chosen event weights, so they cover the rules' cases broadly but do not represent any real person's work. The cost model relies on GitHub's published formula, which GitHub may change, and the benchmark ran on one machine. The agent comparison is a cost model built on measured workloads, not a run of an agent; Section 6.4 lists its limits.

7 Related work

Synchronization and replication. File synchronizers detect updates by comparing each replica with an archive of the last synchronized state [7, 52]; optimistic replication generalizes this, with conflict detection and resolution when both sides change [55, 60], and conflict-free replicated data types avoid conflicts by construction [59]. ferryGL has no conflicts to resolve, because it splits ownership: the GitHub state is authoritative for edges, and the user for everything in between. It also keeps the archive in the destination, in link metadata, rather than on the synchronizing machine. Bidirectional transformations, or lenses, propagate edits of a view back to its source [21]; ferryGL's Linear issues are a lossy, one-way view whose edits never flow back.

Integration and reconciliation. In the vocabulary of enterprise integration patterns [34], ferryGL is a polling consumer and message translator whose receiver is made idempotent by looking up earlier

results by key, here the GitHub URL, the kind of repeated work that distributed systems must make harmless [33]. A proposed `Idempotency-Key` header would let HTTP clients retry creates safely [36]; Linear's API offers a similar means, an identifier the client chooses for each issue or comment it creates [42]. Kubernetes-style controllers reconcile continuously toward a desired state [10]; ferryGL illustrates why that level-triggered model is ill-suited when the destination has an owner of its own, and how recording the last acted-on state turns polling into edge-triggered updates.

AI agents, tools and their cost. Language-model agents that interleave reasoning with tool calls [67] can work in repositories and resolve real GitHub issues [37, 66], so such an agent could do ferryGL's job. But these models are not deterministic in practice, even with settings meant to make them so [6, 51], and they cost tokens on every run. Models can learn to call tools [57], act through executable code [62], and write tools for later use [11], and protocols such as the Model Context Protocol connect agents to them [2]. Systems that combine models with conventional components are increasingly the norm [68]. Cascades that route requests to cheaper models reduce cost [12]; evaluations of agents increasingly report it alongside accuracy [38], and efficiency has long been argued to be a goal in its own right [58]; and a simple, non-agentic pipeline has outperformed open-source software agents on real GitHub issues at a fraction of their cost [65]. That last result is closest in spirit to this paper. Where tool-making agents write a tool for their own later use, ferryGL is a tool written once, tested and shared, and the view it takes is that a routine, rule-based task is better done by a small deterministic program, token-free, which the agent calls when the task comes up.

8 Limitations and future work

- **Order dependence.** The case found in Section 6.1 should be fixed, with a planning test that states the scenario, and the order-independence check kept as a regression test, ideally over random permutations rather than reversal alone.
- **Live measurements.** The agents' costs are estimated and their correctness is not measured; running real agents over the same histories would measure both. The Linear client is tested against Linear's published schema and against fakes, but not yet against a real workspace.
- **Polling and limits of GitHub's API.** Linear catches up only when ferryGL runs; a template for running it on a schedule with GitHub Actions is planned. A search returns at most 1,000 results, the commit scan reads the first 100 commits of each pull request with up to 5 authors each, and ferryGL reads the first 20 issues GitHub says a pull request will close (mentions in its text are not limited). Review requests to a team the user is on do not count as the user's own.
- **Scope and cleanup.** Commenting on an item or being mentioned in it does not bring it into scope; whether either should is an open question. When an item leaves the user's scope, its Linear issue stays as it is: nothing is ever deleted, but a Linear issue can outlive the user's involvement.
- **Retrying creates.** A comment whose creation failed in transit may be posted twice. Choosing the identifier of each new issue and comment on the client, which Linear's API allows [42], would let ferryGL retry creates without that risk.
- **Agent interfaces.** Agents use ferryGL through its command line and read its text output. Machine-readable output, or a Model Context Protocol server [2], would let agents call it without a shell; neither exists yet.

9 Conclusion

ferryGL brings a person's work in other organizations' GitHub repositories into their own Linear workspace, with only their own credentials. It keeps its synchronization state in the destination, in the

metadata of the links between the two systems, and acts on edges in the GitHub state rather than on its level, which gives idempotent reruns without a database and leaves the user's edits alone. A pure planner, a client interface that cannot express the forbidden writes, and eight explicit invariants make it safe to run in repositories that belong to someone else; tens of thousands of simulated runs with failing writes found no violation of them, and one ordering defect worth fixing. By this paper's estimate, an AI agent doing the same job itself would process 91,620 to 217,857 tokens on a run of a 100-item route that finds a few changes, and cost nearly as much on a run that finds none; for a team of 100, across 24 models, that is $596–$59,576 a year, against $59–$6,249 for agents that delegate to ferryGL and nothing on inference for ferryGL alone. Both lessons generalize. A one-way copy into a destination that has an owner of its own can keep its memory in the destination and act on edges. And a routine, rule-based task that agents are asked to repeat is a candidate for token-free automation: written once as a deterministic tool, it costs API requests instead of inference, behaves the same on every run, and leaves the agent's tokens for the work that needs them.

Availability

ferryGL is free software under the MIT license. The source is at https://github.com/ao-batouche/ferryGL, releases are published on PyPI as `ferrygl`, and the documentation is at https://oussamabatouche.com/ferryGL/. This paper describes version 0.2.1. Its source, the evaluation scripts, and the data behind every table and figure are in the repository, under `docs/paper/`, and the paper is archived at doi:10.5281/zenodo.23475998.

Declaration of AI use

AI tools were used to build and maintain ferryGL and to write this paper. AI coding agents were used to build and maintain the software, working under the contributor rules in the repository's `AGENTS.md`, which state the invariants of Section 4. This paper, including its figures, the evaluation scripts and the verification of its references against their sources, was drafted and revised with Anthropic's Claude (Claude Opus 5.5, through Claude Code). The author directed this work, reviewed and edited its output, and takes full responsibility for the software and for the content of this paper. In line with COPE's position, AI tools are not listed as authors [16].

Trademarks

ferryGL is an independent project, not affiliated with or endorsed by Linear or GitHub. Linear and GitHub are trademarks of their respective owners.

References

[1] Alibaba Cloud. Model studio: Qwen3.8-Max and Qwen3.8-Flash. Alibaba Cloud Model Studio Docs, international (Singapore) prices. https://www.alibabacloud.com/help/en/model-studio/qwen3-8-max, 2026. Accessed 2026-10-19.

[2] Anthropic. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol, November 2024. Published 25 November 2024. Accessed 2026-10-19.

[3] Anthropic. Prompt caching. Claude Platform Docs. https://platform.claude.com/docs/en/build-with-claude/prompt-caching, 2026. Accessed 2026-10-19.

[4] Anthropic. Pricing. Claude Platform Docs. https://platform.claude.com/docs/en/about-claude/pricing, 2026. Accessed 2026-10-19.

[5] Artificial Analysis. LLM leaderboard: Output speed and latency. https://artificialanalysis.ai/leaderboards/models, model pages, and methodology at https://artificialanalysis.ai/methodology/

performance-benchmarking. Medians over 72 hours, retrieved 2026-10-19 12:40–12:45 UTC, 2026. Accessed 2026-10-19.

[6] Berk Atıl, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. Nondeterminism of "deterministic" LLM system settings in hosted environments. In Mousumi Akter, Tahiya Chowdhury, Steffen Eger, Christoph Leiter, Juri Opitz, and Erion Çano, editors, *Proceedings of the 5th Workshop on Evaluation and Comparison of NLP Systems (Eval4NLP 2025)*, pages 135–148, Mumbai, India, December 2025. Association for Computational Linguistics. ISBN 979-8-89176-305-0. doi: 10.18653/v1/2025.eval4nlp-1.12. URL https://aclanthology.org/2025.eval4nlp-1.12/.

[7] S. Balasubramaniam and Benjamin C. Pierce. What is a file synchronizer? In *Proceedings of the 4th Annual ACM/IEEE International Conference on Mobile Computing and Networking (MobiCom '98)*, pages 98–108. ACM, October 1998. doi: 10.1145/288235.288261.

[8] Gary Bernhardt. Functional core, imperative shell. Destroy All Software screencast. https://www.destroyallsoftware.com/screencasts/catalog/functional-core-imperative-shell, July 2012. Published 2012-07-12. Accessed 2026-10-19.

[9] Dane Bertram, Amy Voida, Saul Greenberg, and Robert Walker. Communication, collaboration, and bugs: The social nature of issue tracking in small, collocated teams. In *Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work (CSCW '10)*, pages 291–300. ACM, February 2010. doi: 10.1145/1718918.1718972.

[10] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, May 2016. doi: 10.1145/2890784.

[11] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://openreview.net/forum?id=qV83K9d5WB.

[12] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=cSimKw5p6R.

[13] T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, The Hong Kong University of Science and Technology, Hong Kong, 1998. Republished 2020 as arXiv:2002.12543, https://arxiv.org/abs/2002.12543.

[14] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, January 2018. doi: 10.1145/3143561.

[15] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*, pages 268–279. ACM, September 2000. doi: 10.1145/351240.351266.

[16] Committee on Publication Ethics (COPE). Authorship and AI tools. COPE position statement. https://publicationethics.org/cope-position-statements/ai-author, February 2023. First published 13 February 2023. Accessed 2026-10-19.

[17] DeepSeek. Models & pricing. DeepSeek API Docs. https://api-docs.deepseek.com/quick_start/pricing, 2026. Accessed 2026-10-19.

[18] Encode OSS Ltd. HTTPX: The next generation HTTP client for Python. https://www.python-httpx.org/, 2024. Version 0.28.1, released 2024-12-06. Source: https://github.com/encode/httpx. Accessed 2026-10-19.

[19] European Parliament and Council of the European Union. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation). Official Journal of the European Union, L 119, 4 May 2016, pp. 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj, 2016. Accessed 2026-10-19.

[20] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP semantics. Request for Comments 9110, RFC Editor, June 2022. URL https://www.rfc-editor.org/rfc/rfc9110. STD 97. Editors.

[21] J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, 29(3), May 2007. doi: 10.1145/1232420.1232424. Article 17.

[22] GitHub. GitHub Actions billing. GitHub Docs. https://docs.github.com/en/billing/concepts/product-billing/github-actions, 2026. Accessed 2026-10-19.

[23] GitHub. Actions runner pricing. GitHub Docs. https://docs.github.com/en/billing/reference/actions-runner-pricing, 2026. Accessed 2026-10-19.

[24] GitHub. Rate limits and query limits for the GraphQL API. GitHub Docs. https://docs.github.com/en/graphql/overview/rate-limits-and-query-limits-for-the-graphql-api, 2026. Accessed 2026-10-19.

[25] GitHub. Linking a pull request to an issue. GitHub Docs. https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/linking-a-pull-request-to-an-issue, 2026. Accessed 2026-10-19.

[26] GitHub. Public schema. GitHub Docs (GraphQL API). https://docs.github.com/en/graphql/overview/public-schema; schema file https://docs.github.com/public/fpt/schema.docs.graphql, 2026. Accessed 2026-10-19.

[27] GitHub. Searching issues and pull requests. GitHub Docs. https://docs.github.com/en/search-github/searching-on-github/searching-issues-and-pull-requests, 2026. Accessed 2026-10-19.

[28] GitHub. Creating webhooks. GitHub Docs. https://docs.github.com/en/webhooks/using-webhooks/creating-webhooks, 2026. Accessed 2026-10-19.

[29] Google. Gemini developer API pricing. Gemini API Docs. https://ai.google.dev/gemini-api/docs/pricing, with the context caching guide, 2026. Accessed 2026-10-19.

[30] Georgios Gousios, Martin Pinzger, and Arie van Deursen. An exploratory study of the pull-based software development model. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*, pages 345–355. ACM, May 2014. doi: 10.1145/2568225.2568260.

[31] Georgios Gousios, Margaret-Anne Storey, and Alberto Bacchelli. Work practices and challenges in pull-based development: The contributor's perspective. In *Proceedings of the 38th International Conference on Software Engineering (ICSE '16)*, pages 285–296. ACM, May 2016. doi: 10.1145/2884781.2884826.

[32] GraphQL Specification Project. GraphQL specification, September 2025 edition. https://spec.graphql.org/September2025/, September 2025. Accessed 2026-10-19.

[33] Pat Helland. Idempotence is not a medical condition. *Communications of the ACM*, 55(5):56–65, May 2012. doi: 10.1145/2160718.2160734.

[34] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003. ISBN 978-0-321-20068-6.

[35] Jeremy Howard. The /llms.txt file. https://llmstxt.org/, September 2024. Published 3 September 2024; revised in August 2026. Accessed 2026-10-19.

[36] Jayadeba Jena and Sanjay Dalal. The Idempotency-Key HTTP header field. Internet-Draft draft-ietf-httpapi-idempotency-key-header-07, IETF HTTPAPI Working Group. https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/07/, October 2025. Work in progress; expired 18 April 2026. Accessed 2026-10-19.

[37] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[38] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Transactions on Machine Learning Research*, 2025. URL https://openreview.net/forum?id=Zy4uFzMviZ.

[39] Linear. Attachments. Linear Developers. https://linear.app/developers/attachments, 2026. Accessed 2026-10-19.

[40] Linear. Getting started. Linear Developers (GraphQL API). https://linear.app/developers/graphql, 2026. Accessed 2026-10-19.

[41] Linear. GitHub. Linear Docs. https://linear.app/docs/github, 2026. Accessed 2026-10-19.

[42] Linear. Linear GraphQL schema (`packages/sdk/src/schema.graphql`). GitHub repository linear/linear. https://github.com/linear/linear/blob/master/packages/sdk/src/schema.graphql, 2026. Last changed in commit 9dd2ed8 (2026-10-15). Accessed 2026-10-19.

[43] Make. GitHub and Linear integration. https://www.make.com/en/integrations/github/linear, 2026. Accessed 2026-10-19.

[44] André N. Meyer, Laura E. Barton, Gail C. Murphy, Thomas Zimmermann, and Thomas Fritz. The work life of developers: Activities, switches and perceived productivity. *IEEE Transactions on Software Engineering*, 43 (12):1178–1193, December 2017. doi: 10.1109/TSE.2017.2656886.

[45] MiniMax. Pricing overview. MiniMax Platform Docs. https://platform.minimax.io/docs/pricing/overview, 2026. Accessed 2026-10-19.

[46] Mistral AI. Pricing. Mistral Docs. https://docs.mistral.ai/inference/pricing, 2026. Accessed 2026-10-19.

[47] Moonshot AI. Model inference pricing. Kimi Open Platform Docs. https://platform.kimi.ai/docs/pricing/chat, 2026. Accessed 2026-10-19.

[48] Ben Moseley and Peter Marks. Out of the tar pit. https://curtclifton.net/papers/MoseleyMarks06a.pdf, February 2006. Dated February 6, 2006. Accessed 2026-10-19.

[49] n8n. GitHub and Linear integration. https://n8n.io/integrations/github/and/linear/, 2026. Accessed 2026-10-19.

[50] OpenAI. Pricing. OpenAI API Docs. https://developers.openai.com/api/docs/pricing, with the model pages and the prompt caching guide, 2026. Accessed 2026-10-19.

[51] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):1–28, February 2025. doi: 10.1145/3697010.

[52] Benjamin C. Pierce and Jérôme Vouillon. What's in Unison? A formal specification and reference implementation of a file synchronizer. Technical Report MS-CIS-03-36, Department of Computer and Information Science, University of Pennsylvania, February 2004. URL https://www.cis.upenn.edu/~bcpierce/papers/unisonspec.pdf.

[53] Tom Preston-Werner, Pradyun Gedam, et al. TOML v1.0.0. https://toml.io/en/v1.0.0, January 2021. Released 2021-01-11. Accessed 2026-10-19.

[54] Vivek Rau. Eliminating toil. In Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy, editors, *Site Reliability Engineering: How Google Runs Production Systems*, chapter 5. O'Reilly Media, 2016. ISBN 978-1-491-92912-4. URL https://sre.google/sre-book/eliminating-toil/.

[55] Yasushi Saito and Marc Shapiro. Optimistic replication. *ACM Computing Surveys*, 37(1):42–81, March 2005. doi: 10.1145/1057977.1057980.

[56] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.

[57] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551. Curran Associates, Inc., 2023. doi: 10.52202/075280-2997.

[58] Roy Schwartz, Jesse Dodge, Noah A. Smith, and Oren Etzioni. Green AI. *Communications of the ACM*, 63(12): 54–63, 2020. doi: 10.1145/3381831.

[59] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In Xavier Défago, Franck Petit, and Vincent Villain, editors, *Stabilization, Safety, and Security of Distributed Systems: 13th International Symposium, SSS 2011, Grenoble, France, October 10–12, 2011. Proceedings*, volume 6976 of *Lecture Notes in Computer Science*, pages 386–400, Berlin, Heidelberg, 2011. Springer. doi: 10.1007/978-3-642-24550-3_29.

[60] Douglas B. Terry, Marvin M. Theimer, Karin Petersen, Alan J. Demers, Mike J. Spreitzer, and Carl H. Hauser. Managing update conflicts in Bayou, a weakly connected replicated storage system. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP '95)*, pages 172–183. ACM, 1995. doi: 10.1145/224056.224070.

[61] Unito. How to install and configure Unito for GitHub. Unito Help Center. https://docs.unito.io/how-to-install-and-configure-unito-for-github/, 2026. Page last updated Oct 28, 2025. Accessed 2026-10-19.

[62] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 50208–50232. PMLR, 2024. URL https://proceedings.mlr.press/v235/wang24h.html.

[63] Austin Wright and Henry Andrews. JSON schema: A media type for describing JSON documents. Internet-Draft draft-handrews-json-schema-01 (JSON Schema Draft-07 Core). https://datatracker.ietf.org/doc/draft-handrews-json-schema/01/, March 2018. Accessed 2026-10-19.

[64] xAI. Models and pricing. xAI Docs. https://docs.x.ai/developers/pricing, 2026. Accessed 2026-10-19.

[65] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software

engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754.

[66] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652. Curran Associates, Inc., 2024. doi: 10.52202/079017-1601.

[67] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

[68] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. The shift from models to compound AI systems. The Berkeley Artificial Intelligence Research Blog. https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/, February 2024. Published 18 February 2024. Accessed 2026-10-19.

[69] Z.ai. Pricing. Z.ai Docs. https://docs.z.ai/guides/overview/pricing, 2026. Accessed 2026-10-19.

[70] Zapier. GitHub Linear integration. https://zapier.com/apps/github/integrations/linear, 2026. Accessed 2026-10-19.

A Results per model

Table 7. The values behind Figure 8, for one user's route of 100 items on a run that finds 4 changes: tokens per run of an agent doing the job itself (plausible), dollars a year for a team of 100 at two runs a workday (lean to plausible; one user costs a hundredth as much), and seconds per run (plausible; — where no speed was measured). [a] Price from 1 December 2026 (Gemini 4 Argon) or 1 January 2027 (Gemini 3.8 Flash); half until then. [b] Peak-hour price; other hours cost half. [c] Preview, at its regular price. [d] Released on 19 October 2026. [e] $0 on own machines; the upper figure on paid Actions minutes.

		Tokens	Dollars a year, team of 100		Seconds per run	
Provider	Model	per run	Itself	Delegating	Itself	Delegating
Anthropic	Claude Fable 5.1	217,857	$32,036–$53,365	$2,113–$5,817	62	28
	Claude Opus 5.5	217,857	$13,093–$22,174	$854–$2,338	96	50
	Claude Sonnet 5.5	217,857	$6,547–$11,087	$427–$1,169	34	22
	Claude Haiku 5.5	217,857	$341–$596	$22–$59	70	44
OpenAI	GPT-6 Astra	217,857	$34,126–$59,576	$2,983–$6,249	64	28
	GPT-6.1 Sol	217,857	$6,547–$11,087	$597–$1,243	81	31
	GPT-6 Luna	217,857	$341–$596	$30–$62	29	22
Google	Gemini 4 Argon[a]	217,857	$11,869–$19,926	$1,131–$2,653	—	—
	Gemini 3.8 Flash[a]	217,857	$4,647–$8,080	$424–$995	164	91
	Gemini 3.5 Flash-Lite	217,857	$1,034–$1,862	$87–$259	54	40
xAI	Grok 4.7	217,857	$6,103–$11,760	$327–$802	83	43
	Grok 4.3	217,857	$3,311–$6,051	$161–$377	—	—
DeepSeek	DeepSeek V4 Pro[b]	220,799	$3,412–$5,464	$190–$498	39	25
	DeepSeek V4.1 Flash[b]	220,799	$811–$1,297	$51–$140	75	48
Moonshot AI	Kimi K3	212,295	$8,814–$15,304	$608–$1,691	368	175
	Kimi K2.7 Code	212,295	$2,942–$5,396	$181–$479	299	147
Alibaba	Qwen3.8 Max	235,014	$6,201–$10,429	$463–$844	415	191
	Qwen3.8 Flash	235,014	$460–$764	$35–$65	299	142
Z.ai	GLM-5.3	217,857	$4,199–$7,545	$226–$569	171	90
	GLM-5.3-Flash	217,857	$461–$835	$25–$64	206	105
MiniMax	MiniMax-M3	212,886	$922–$1,690	$56–$146	142	77
	MiniMax-M3.1-Flash[d]	217,857	$418–$740	$26–$71	—	—
Mistral AI	Mistral Large 4[c]	217,857	$3,608–$6,370	$207–$532	214	108
	Mistral Small 4	240,590	$499–$815	$27–$71	25	21
ferryGL alone		0	$0–$317[e]		11	

Table 8. Characters per token of the text an agent reads and writes on a route of 100 items, measured with each public tokenizer of a model family in the comparison. OpenAI's `o200k_base`, used by its GPT-5 models, stands in for a proprietary tokenizer; GPT-6's is not published.

Tokenizer	JSON tool data	Instructions	ferryGL's output
Mistral Small 4	2.38	3.93	3.48
Qwen3.8 Max, Qwen3.8 Flash	2.50	3.89	3.48
DeepSeek V4 Pro, DeepSeek V4.1 Flash	2.75	4.00	3.68
GLM-5.3, GLM-5.3-Flash	2.81	4.02	3.70
OpenAI o200k_base	2.86	4.02	3.66
MiniMax-M3	2.92	4.05	3.66
Kimi K3, Kimi K2.7 Code	2.93	4.06	3.66

B Reproducing the evaluation

From a checkout of the repository, with uv installed:

```
uv sync
uv run python docs/paper/evaluation/simulate.py   # writes data/simulation.*
uv run python docs/paper/evaluation/costs.py      # writes data/github-cost.csv
uv run python docs/paper/evaluation/bench.py      # writes data/planner-time.csv
uv run --with tiktoken --with tokenizers --with huggingface_hub \
  python docs/paper/evaluation/count_tokens.py     # writes data/tokenizers.csv
uv run python docs/paper/evaluation/agent_cost.py # writes data/agent-*
uv run python docs/paper/evaluation/compare_models.py  # writes data/models*
make -C docs/paper                                # builds the PDF with latexmk
```

The simulation is deterministic for a given seed (20261019 by default, one scenario per consecutive seed), and exits with status 1 while any property fails, as order independence does in this version. The cost experiment asserts that its request counts match the model and that ferryGL sent no mutation. `agent_cost.py` and `compare_models.py` call no model; prices, speeds and their sources are in `evaluation/models.py`. Planner timings vary with the machine; the data was measured as described in Section 6.3.